ELO320 - Estructuras de Datos y Algoritmos
Análisis de Complejidad
Insertion Sort

Dr. Nicolás Gálvez R.
nicolas.galvez@usm.cl

Ing. Civil Telemática - Departamento de Electrónica

1er Semestre 2025

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Complejidad en Algoritmos

¿Cómo podemos comparar dos algoritmos que resuelven un mismo problema?

- Opción 1: programar ambos y ver cuántos recursos computacionales consumen. Pero:
  - Poco eficiente en costo $\rightarrow$ sólo uno será utilizado.
  - Algún algoritmo puede ser mal implementado $\rightarrow$ se subestima.
  - Casos de prueba pueden favorecer a algún algoritmo en específico.
  - Ambos algoritmos pueden ser muy costosos de implementar.
- Opción 2: Análisis de Algoritmo $\rightarrow$ Análisis de Tiempo de Ejecución.
- Opción 3: Análisis Asintótico.

Complejidad en Algoritmos

¿Cómo podemos comparar dos algoritmos que resuelven un mismo problema?

- Opción 1: programar ambos y ver cuántos recursos computacionales consumen. Pero:
  - Poco eficiente en costo → sólo uno será utilizado.
  - Algún algoritmo puede ser mal implementado → se subestima.
  - Casos de prueba pueden favorecer a algún algoritmo en específico.
  - Ambos algoritmos pueden ser muy costosos de implementar.
- Opción 2: Análisis de Algoritmo → Análisis de Tiempo de Ejecución.
- Opción 3: Análisis Asintótico.

Complejidad en Algoritmos

¿Cómo podemos comparar dos algoritmos que resuelven un mismo problema?

- Opción 1: programar ambos y ver cuántos recursos computacionales consumen. Pero:
  - Poco eficiente en costo $\rightarrow$ sólo uno será utilizado.
  - Algún algoritmo puede ser mal implementado $\rightarrow$ se subestima.
  - Casos de prueba pueden favorecer a algún algoritmo en específico.
  - Ambos algoritmos pueden ser muy costosos de implementar.
- Opción 2: Análisis de Algoritmo $\rightarrow$ Análisis de Tiempo de Ejecución.
- Opción 3: Análisis Asintótico.

Complejidad en Algoritmos

¿Cómo podemos comparar dos algoritmos que resuelven un mismo problema?

- Opción 1: programar ambos y ver cuántos recursos computacionales consumen. Pero:
  - Poco eficiente en costo $\rightarrow$ sólo uno será utilizado.
  - Algún algoritmo puede ser mal implementado $\rightarrow$ se subestima.
  - Casos de prueba pueden favorecer a algún algoritmo en específico.
  - Ambos algoritmos pueden ser muy costosos de implementar.
- Opción 2: Análisis de Algoritmo $\rightarrow$ Análisis de Tiempo de Ejecución.
- Opción 3: Análisis Asintótico.

Complejidad en Algoritmos

¿Cómo podemos comparar dos algoritmos que resuelven un mismo problema?

- Opción 1: programar ambos y ver cuántos recursos computacionales consumen. Pero:
  - Poco eficiente en costo $\rightarrow$ sólo uno será utilizado.
  - Algún algoritmo puede ser mal implementado $\rightarrow$ se subestima.
  - Casos de prueba pueden favorecer a algún algoritmo en específico.
  - Ambos algoritmos pueden ser muy costosos de implementar.
- Opción 2: Análisis de Algoritmo $\rightarrow$ Análisis de Tiempo de Ejecución.
- Opción 3: Análisis Asintótico.

Complejidad en Algoritmos

¿Cómo podemos comparar dos algoritmos que resuelven un mismo problema?

- Opción 1: programar ambos y ver cuántos recursos computacionales consumen. Pero:
  - Poco eficiente en costo $\rightarrow$ sólo uno será utilizado.
  - Algún algoritmo puede ser mal implementado $\rightarrow$ se subestima.
  - Casos de prueba pueden favorecer a algún algoritmo en específico.
  - Ambos algoritmos pueden ser muy costosos de implementar.
- Opción 2: Análisis de Algoritmo $\rightarrow$ Análisis de Tiempo de Ejecución.
- Opción 3: Análisis Asintótico.

Complejidad en Algoritmos

¿Cómo podemos comparar dos algoritmos que resuelven un mismo problema?

- Opción 1: programar ambos y ver cuántos recursos computacionales consumen. Pero:
  - Poco eficiente en costo $\rightarrow$ sólo uno será utilizado.
  - Algún algoritmo puede ser mal implementado $\rightarrow$ se subestima.
  - Casos de prueba pueden favorecer a algún algoritmo en específico.
  - Ambos algoritmos pueden ser muy costosos de implementar.
- Opción 2: Análisis de Algoritmo $\rightarrow$ Análisis de Tiempo de Ejecución.
- Opción 3: Análisis Asintótico.

Complejidad en Algoritmos

¿Cómo podemos comparar dos algoritmos que resuelven un mismo problema?

- Opción 1: programar ambos y ver cuántos recursos computacionales consumen. Pero:
  - Poco eficiente en costo $\rightarrow$ sólo uno será utilizado.
  - Algún algoritmo puede ser mal implementado $\rightarrow$ se subestima.
  - Casos de prueba pueden favorecer a algún algoritmo en específico.
  - Ambos algoritmos pueden ser muy costosos de implementar.
- Opción 2: Análisis de Algoritmo $\rightarrow$ Análisis de Tiempo de Ejecución.
- Opción 3: Análisis Asintótico.

Complejidad en Algoritmos

- Medir eficiencia $\rightarrow$ entrada de tamaño creciente ($n \rightarrow \infty$).
- Técnicas de estimación del costo.
- Definir costos de un algoritmo antes de implementarlo.

Algoritmo

Entrada $\rightarrow$ Procedimiento $\rightarrow$ Salida.

Se miden por recursos utilizados:

- Tiempo.
- Memoria.
- Ancho de Banda usado.

Complejidad en Algoritmos

- Medir eficiencia $\rightarrow$ entrada de tamaño creciente ($n \rightarrow \infty$).
- Técnicas de estimación del costo.
- Definir costos de un algoritmo antes de implementarlo.

Algoritmo

Entrada $\rightarrow$ Procedimiento $\rightarrow$ Salida.

Se miden por recursos utilizados:

- **Tiempo.**
- Memoria.
- Ancho de Banda usado.

Complejidad en Algoritmos

El tiempo de un algoritmo varía según donde sea implementado. Por esta razón, nos referiremos a los siguientes elementos (normalizados):

- Cantidad de accesos a la memoria.
- Número de comparaciones
- Cantidad de veces que se ejecuta un ciclo.

Esto se resume en:

Complejidad en tiempo

Nº de operaciones realizadas $\times$ **Costo de cada operación**.

Complejidad en Algoritmos

Complejidad en tiempo

Nº de operaciones realizadas $\times$ **Costo de cada operación**.

Utilizaremos una medida estándar (*ceteris paribus*):

$$\textbf{Costo de cada operación} \rightarrow \textbf{1}.$$

Esto debido a que una plataforma con mejor características producirá que un mismo algoritmo tenga mejor desempeño respecto a otra plataforma.

Tiempo de ejecución de un Algoritmo

Definiremos $T(n)$ como la función de tiempo de ejecución de un algoritmo con input n, y la expresaremos en función a la cantidad de instrucciones.

Costo de las operaciones en unidades de tiempo [ut]:

- Instrucciones básicas: 1
- Secuencia de n instrucciones: $c \times n = n$ (c: cte. de costo)
- if-else: $T(\text{condición}) + \max\{T(\text{if}), T(\text{else})\}$
- while: $(n+1) \times T(\text{condición}) + n \times T(\text{while})$

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + (n \times 1) = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + (n \times 1) = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + (n \times 1) = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + (n \times 1) = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + (n \times 1) = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + (n \times 1) = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + (n \times 1) = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + (n \times 1) = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + (n \times 1) = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + (n \times 1) = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + (n \times 1) = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + (n \times 1) = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + (n \times 1) = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + (n \times 1) = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + (n \times 1) = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + (n \times 1) = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Complejidad Algorítmica vs Entorno de ejecución.

Los primeros algoritmos de ordenamiento que veremos son:

- **Insert Sort**: $T_1(n) = c_1 \times n^2$
- **Merge Sort**: $T_2(n) = c_2 \times n \times \log_2 n$

Supongamos que el PC-A está ejecutando **Insert Sort**, que el PC-B está ejecutando **Merge Sort** y que ambos deben ordenar un arreglo de 10 millones de enteros. Además, el PC-A es de última generación y ejecuta 10 mil millones de instrucciones por segundo y el PC-B tiene 10 años de antigüedad y sólo procesa 10 millones de instrucciones por segundo. Incluyamos lo siguiente:

- Paco Web programó **Insert Sort** en el PC-A usando lenguaje de máquina (assembly) resultando en un costo de $2 \times n^2$ instrucciones para ordenar n números.
- El Corxea implementa **Merge Sort**, en el PC-B usando COBOL, con un compilador antiguo e ineficiente, y el código resultante toma $50 \times n \times \log_2 n$ instrucciones para ordenar n números.

Complejidad Algorítmica vs Entorno de ejecución.

- PC-A:

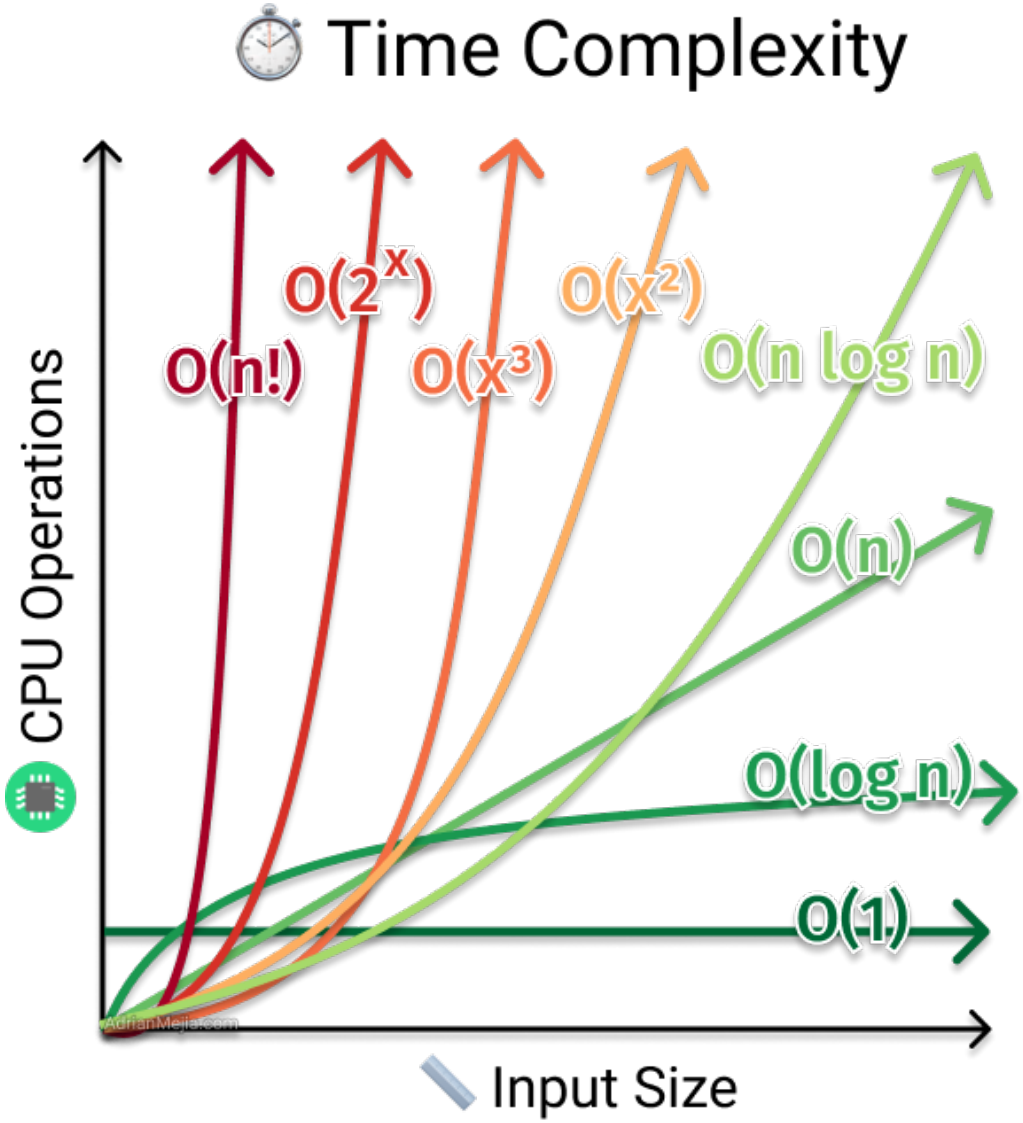
$$\frac{2 \times (10^7)^2}{10^{10}} = 20000\ [s] \quad (> 5{,}5[h])$$

- PC-B:

$$\frac{50 \times 10^7 \times \log_2 10^7}{10^7} \approx 1163\ [s] \quad (< 20[min])$$

PC-B completa su algoritmo 17 veces más rápido que el algoritmo usado en PC-A, siendo el segundo entorno de ejecución 1000 veces más lento que el primero.

Complejidad Algorítmica vs Entorno de ejecución.



Análisis Asintótico

Una vez entendido el análisis de tiempo buscaremos análisis asintótico de los algoritmos. En la práctica significa concentrarse en el término predominante en $T(n)$, y luego encontrar cotas de rendimiento.

Existen tres notaciones asintóticas generalmente usadas:

- Big O ($\mathcal{O}$): Cota superior asintótica, vale decir, cota al peor desempeño.
- Big Omega (Ω): Cota inferior asintótica, vale decir, cota al mejor desempeño.
- Big Theta (Θ): Cota compuesta, vale decir, cota al rendimiento promedio.

Comúnmente, se utiliza la notación en peor escenario, vale decir, Big O ($\mathcal{O}$).

Análisis Asintótico: Big O ($\mathcal{O}$)

Cota asintótica que limita el peor desempeño de un algoritmo.

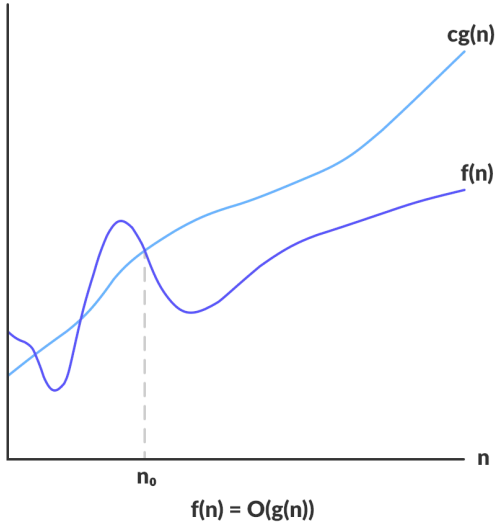
Big O ($\mathcal{O}$)

Sean f y g dos funciones en $\mathbb{R}$, diremos que:

$$f(n) = \mathcal{O}(g(n)) \iff f(n) \in \mathcal{O}(g(n))$$

Donde:

$$\mathcal{O}(g(n)) = \{f(n) \mid \exists c \in \mathbb{R}, f(n) \leq c \times g(n);\ \forall n \geq n_0\}$$



Análisis Asintótico: Big Omega (Ω)

Cota asintótica que limita el mejor desempeño de un algoritmo.

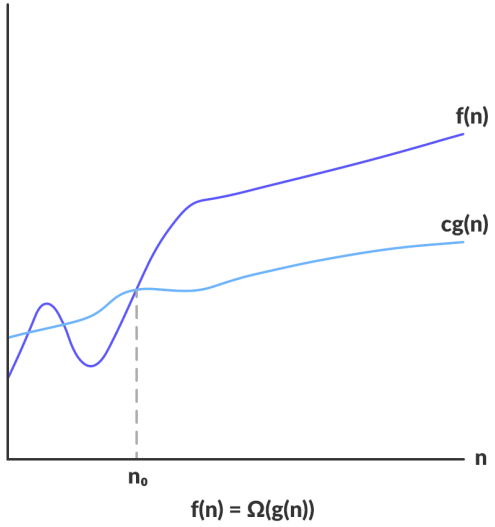
Big Omega (Ω)

Sean f y g dos funciones en $\mathbb{R}$, diremos que:

$$f(n) = \Omega(g(n)) \iff f(n) \in \Omega(g(n))$$

Donde:

$$\Omega(g(n)) = \{f(n) \mid \exists c \in \mathbb{R}, c \times g(n) \leq f(n);\ \forall n \geq n_0\}$$



Análisis Asintótico: Big Theta (Θ)

Cota asintótica que limita el desempeño promedio de un algoritmo.

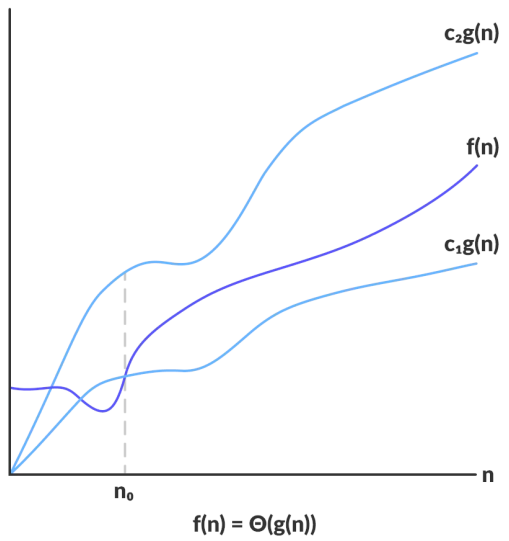
Big Theta (Θ)

Sean f y g dos funciones en $\mathbb{R}$, diremos que:

$$f(n) = \Theta(g(n)) \iff f(n) \in \Theta(g(n))$$

Donde:

$$\Theta(g(n)) = \{f(n) \mid \exists (c_1, c_2) \in \mathbb{R}^2, c_1 \times g(n) \leq f(n) \leq c_2 \times g(n);\ \forall n \geq n_0\}$$



Análisis Asintótico

¿Cuál es la complejidad del siguiente algoritmo

```
int suma(int n) {
        int sum = 0, i, j;
        for (i=0; i < n; i++)
                for (j=0; j < n; j++)
                        sum++;
        return sum;
}
```

- Costo total $f(n) = 3n^2 + 4n + 6$.
- $g(n) = n^2$, termino del grado superior.
- Cota superior: $\texttt{suma} = \mathcal{O}(n^2) \rightarrow f(n) \leq 10 \times g(n)$
- Cota inferior: $\texttt{suma} = \Omega(n^2) \rightarrow 1 \times g(n) \leq f(n)$
- Cota promedio: $\texttt{suma} = \Theta(n^2) \rightarrow 1 \times g(n) \leq f(n) \leq 10 \times g(n)$

Análisis Asintótico

¿Cuál es la complejidad del siguiente algoritmo

```
int suma(int n) {
        int sum = 0, i, j;
        for (i=0; i < n; i++)
                for (j=0; j < n; j++)
                        sum++;
        return sum;
}
```

- Costo total $f(n) = 3n^2 + 4n + 6$.
- $g(n) = n^2$, termino del grado superior.
- Cota superior: $\texttt{suma} = \mathcal{O}(n^2) \rightarrow f(n) \leq 10 \times g(n)$
- Cota inferior: $\texttt{suma} = \Omega(n^2) \rightarrow 1 \times g(n) \leq f(n)$
- Cota promedio: $\texttt{suma} = \Theta(n^2) \rightarrow 1 \times g(n) \leq f(n) \leq 10 \times g(n)$

Análisis Asintótico

¿Cuál es la complejidad del siguiente algoritmo

```
int suma(int n) {
        int sum = 0, i, j;
        for (i=0; i < n; i++)
                for (j=0; j < n; j++)
                        sum++;
        return sum;
}
```

- Costo total $f(n) = 3n^2 + 4n + 6$.
- $g(n) = n^2$, termino del grado superior.
- Cota superior: $\texttt{suma} = \mathcal{O}(n^2) \rightarrow f(n) \leq 10 \times g(n)$
- Cota inferior: $\texttt{suma} = \Omega(n^2) \rightarrow 1 \times g(n) \leq f(n)$
- Cota promedio: $\texttt{suma} = \Theta(n^2) \rightarrow 1 \times g(n) \leq f(n) \leq 10 \times g(n)$

Análisis Asintótico

¿Cuál es la complejidad del siguiente algoritmo

```
int suma(int n) {
        int sum = 0, i, j;
        for (i=0; i < n; i++)
                for (j=0; j < n; j++)
                        sum++;
        return sum;
}
```

- Costo total $f(n) = 3n^2 + 4n + 6$.
- $g(n) = n^2$, termino del grado superior.
- Cota superior: $\texttt{suma} = \mathcal{O}(n^2) \rightarrow f(n) \leq 10 \times g(n)$
- Cota inferior: $\texttt{suma} = \Omega(n^2) \rightarrow 1 \times g(n) \leq f(n)$
- Cota promedio: $\texttt{suma} = \Theta(n^2) \rightarrow 1 \times g(n) \leq f(n) \leq 10 \times g(n)$

Análisis Asintótico

¿Cuál es la complejidad del siguiente algoritmo

```
int suma(int n) {
        int sum = 0, i, j;
        for (i=0; i < n; i++)
                for (j=0; j < n; j++)
                        sum++;
        return sum;
}
```

- Costo total $f(n) = 3n^2 + 4n + 6$.
- $g(n) = n^2$, termino del grado superior.
- Cota superior: $\texttt{suma} = \mathcal{O}(n^2) \rightarrow f(n) \leq 10 \times g(n)$
- Cota inferior: $\texttt{suma} = \Omega(n^2) \rightarrow 1 \times g(n) \leq f(n)$
- Cota promedio: $\texttt{suma} = \Theta(n^2) \rightarrow 1 \times g(n) \leq f(n) \leq 10 \times g(n)$

Análisis Asintótico

¿Cuál es la complejidad del siguiente algoritmo

```
int suma(int n) {
        int sum = 0, i, j;
        for (i=0; i < n; i++)
                for (j=0; j < n; j++)
                        sum++;
        return sum;
}
```

- Costo total $f(n) = 3n^2 + 4n + 6$.
- $g(n) = n^2$, termino del grado superior.
- Cota superior: $\texttt{suma} = \mathcal{O}(n^2) \rightarrow f(n) \leq 10 \times g(n)$
- Cota inferior: $\texttt{suma} = \Omega(n^2) \rightarrow 1 \times g(n) \leq f(n)$
- Cota promedio: $\texttt{suma} = \Theta(n^2) \rightarrow 1 \times g(n) \leq f(n) \leq 10 \times g(n)$

Análisis Asintótico

¿Cuál es la complejidad del siguiente algoritmo

```
int suma(int n) {
        int sum = 0, i, j;
        for (i=0; i < n; i++)
                for (j=0; j < n; j++)
                        sum++;
        return sum;
}
```

- Costo total $f(n) = 3n^2 + 4n + 6$.
- $g(n) = n^2$, termino del grado superior.
- Cota superior: $\texttt{suma} = \mathcal{O}(n^2) \rightarrow f(n) \leq 10 \times g(n)$
- Cota inferior: $\texttt{suma} = \Omega(n^2) \rightarrow 1 \times g(n) \leq f(n)$
- Cota promedio: $\texttt{suma} = \Theta(n^2) \rightarrow 1 \times g(n) \leq f(n) \leq 10 \times g(n)$

Análisis Asintótico

¿Cuál es la complejidad del siguiente algoritmo

```
int suma(int n) {
        int sum = 0, i, j;
        for (i=0; i < n; i++)
                for (j=0; j < n; j++)
                        sum++;
        return sum;
}
```

- Costo total $f(n) = 3n^2 + 4n + 6$.
- $g(n) = n^2$, termino del grado superior.
- Cota superior: $\texttt{suma} = \mathcal{O}(n^2) \rightarrow f(n) \leq 10 \times g(n)$
- Cota inferior: $\texttt{suma} = \Omega(n^2) \rightarrow 1 \times g(n) \leq f(n)$
- Cota promedio: $\texttt{suma} = \Theta(n^2) \rightarrow 1 \times g(n) \leq f(n) \leq 10 \times g(n)$

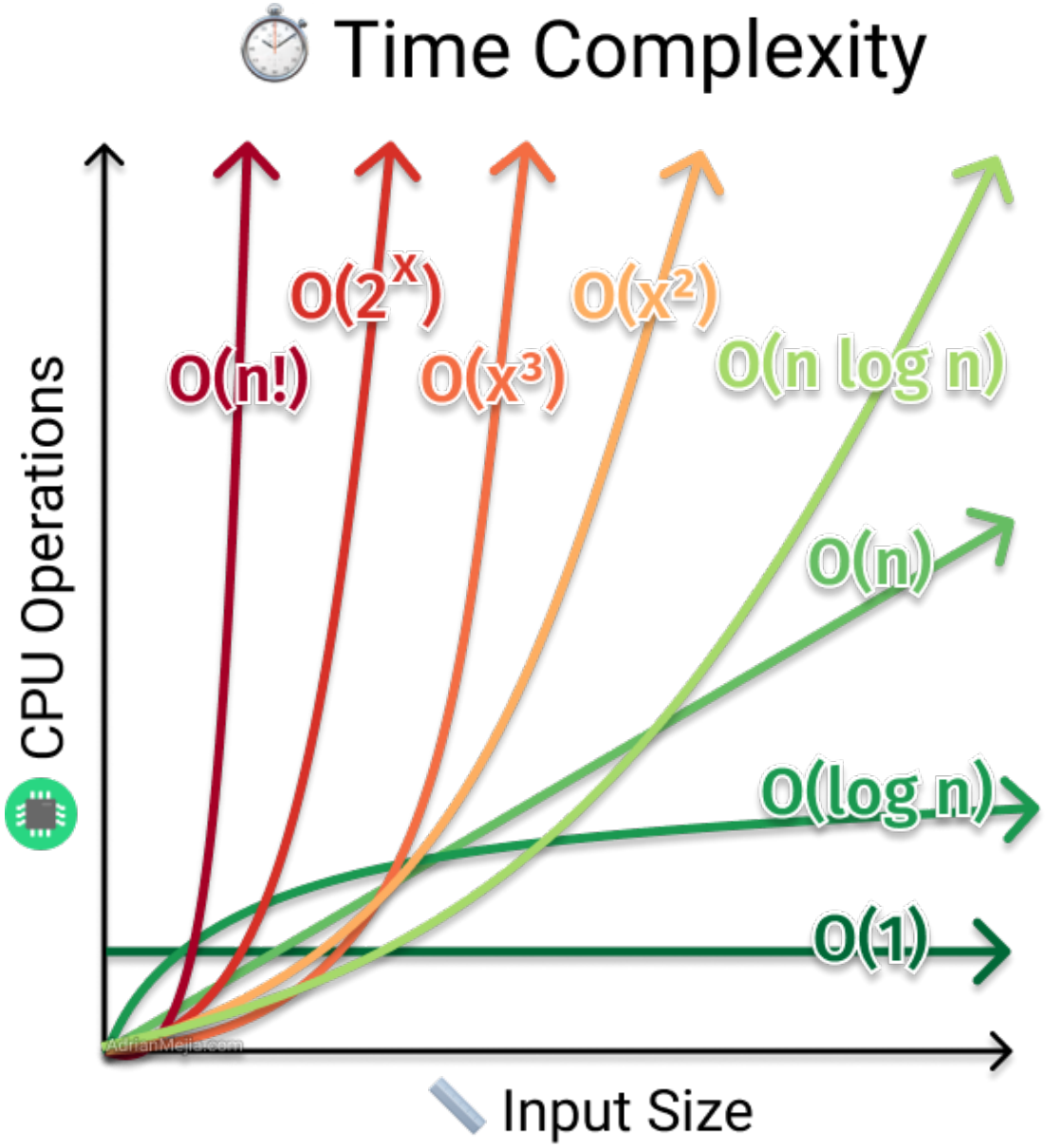
Análisis Asintótico

¿Cuál es la complejidad del siguiente algoritmo

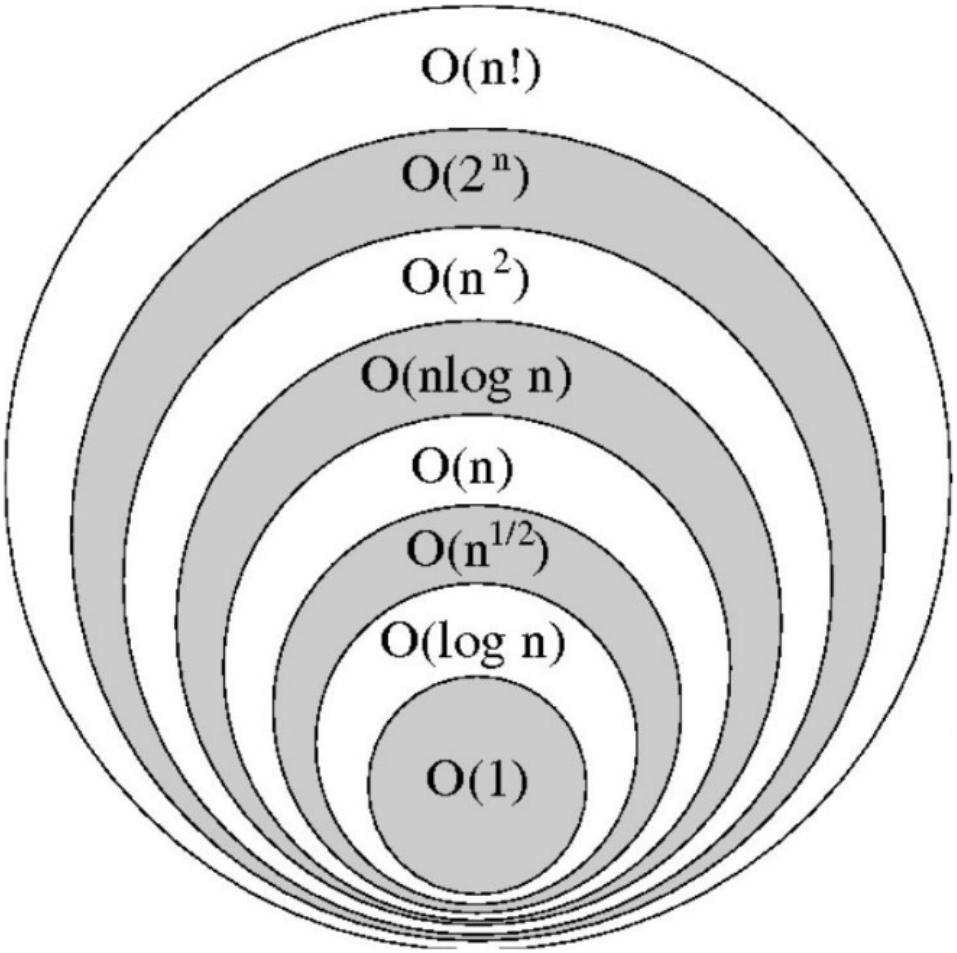
```
int suma(int n) {
        int sum = 0, i, j;
        for (i=0; i < n; i++)
                for (j=0; j < n; j++)
                        sum++;
        return sum;
}
```

- Costo total $f(n) = 3n^2 + 4n + 6$.
- $g(n) = n^2$, termino del grado superior.
- Cota superior: $\texttt{suma} = \mathcal{O}(n^2) \rightarrow f(n) \leq 10 \times g(n)$
- Cota inferior: $\texttt{suma} = \Omega(n^2) \rightarrow 1 \times g(n) \leq f(n)$
- Cota promedio: $\texttt{suma} = \Theta(n^2) \rightarrow 1 \times g(n) \leq f(n) \leq 10 \times g(n)$

Análisis Asintótico

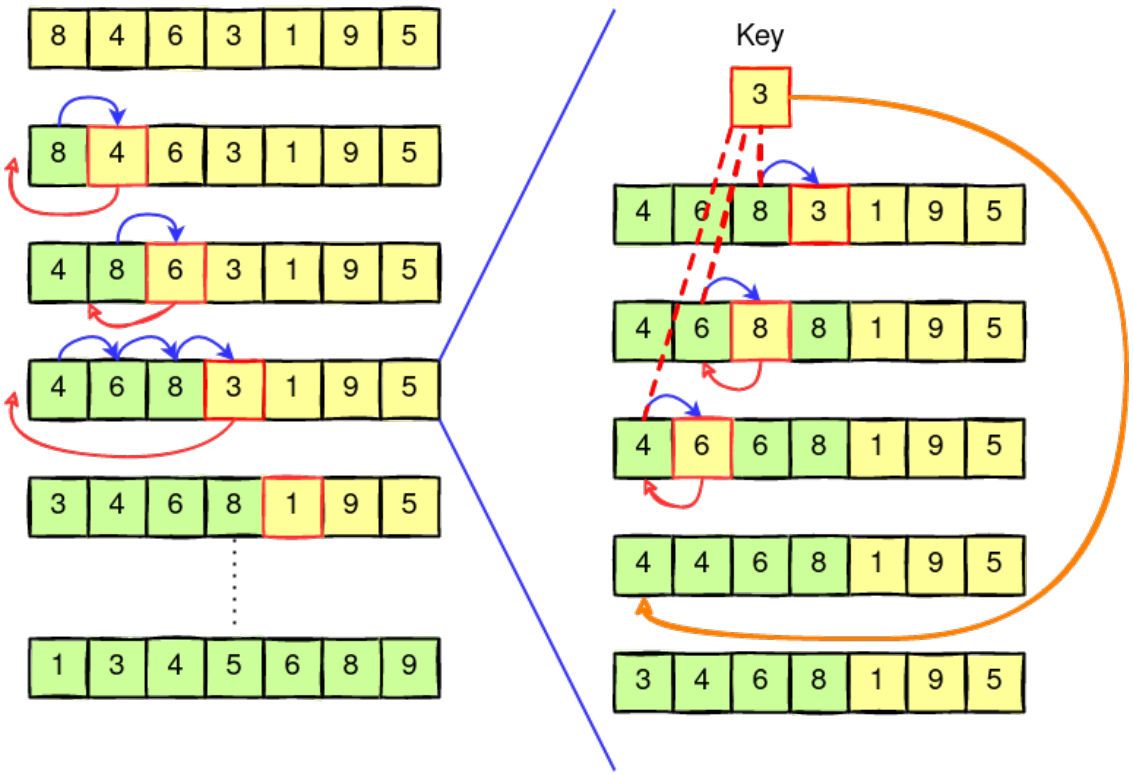


Análisis Asintótico



InsertSort

Es un método de ordenamiento de elementos, en el cual a través de la visitas de subconjuntos, un elemento es **insertado** en la posición ordenada que le corresponde.



InsertSort

¿Cuál es la complejidad del siguiente algoritmo

```
void insertionSort(int arr[], int n){
        int i, key, j;
        for (i = 1; i < n; i++) {
                key = arr[i];
                j = i - 1;
                while (j >= 0 && arr[j] > key) { //esto puede ser un for
                        arr[j + 1] = arr[j];
                        j--;
                }
                arr[j + 1] = key;
        }
}
```

InsertSort: Complejidad

- Cota Superior: $\mathcal{O}(n^2) \rightarrow$ Caso peor: todo invertido.
- Cota Inferior: $\Omega(n) \rightarrow$ Mejor caso: todo ordenado.
- Cota Promedio: $\Theta(n^2) \rightarrow$ Caso promedio para arreglos aleatorios.

Extrapolando a otros algoritmos, ¿Por qué generalmente se usa $\mathcal{O}$ y no Θ? Debido a que en muchos algoritmos, Θ no es sostenible para TODOS los casos.

Ejercicios

- Revisite las implementaciones de listas, pilas, colas y árboles y determine la complejidad de sus métodos.
- Mostrar la evolución de Insert Sort en el arreglo A = 31, 41, 59, 26, 41, 58
- Reescribir Insert Sort para ordenamiento de datos de manera decreciente.
- Expresar la ecuación $n^3/1000 - 100n^2 - 100n + 3$ en términos de notación asintótica.
- Considere el problema de sumar dos números enteros expresados base binaria, almacenados en dos arreglos A y B de n elementos (bits) cada uno. La suma de los dos enteros se debe almacenar en forma binaria en un arreglo C de (n+1) elementos. Escriba una función en C para la suma los dos enteros. ¿Qué complejidad tiene su función? Compárela con la de sus compañeros.